

CHAPTER 4

能否用於電路合成
的 Verilog 語法

Verilog

在本章將為各位介紹：不能用於電路合成的 Verilog 語法，以及能用於電路合成的 Verilog 語法之說明。



4.1 不能用於電路合成的 Verilog 語法

以下的這些「敘述」、「運算子」、「邏輯閘型態」及「建構方式」只能適用於電路的模擬 (Simulation)，並不能用於電路合成 (Synthesis) 的描述電路中。

4.1.1 不能用於電路合成的「敘述」

- delay 敘述。
- repeat 敘述。
- wait 敘述。
- join 敘述。
- time 敘述。
- force 敘述。
- primitive 敘述。
- case identity, not identity operations 敘述。
- rtran, tranif0, tranif1, rtranif0, rtranif1 敘述。
- initial 敘述。
- forever 敘述。
- fork 敘述。
- event 敘述。
- deassign 敘述。
- release 敘述。

4.1.2 不能用於電路合成的「運算子」

- === (連續三個 = 號) 運算子。

■ `!==(一個 ! 號接著二個 = 號)` 運算子。

■ 除法運算子(`/`, division)。

■ 取餘數運算子(`%`, modulus)。

例：在下面這個例子的敘述不能由 Synopsys 來做電路合成 (Synthesis)，因為 Synopsys 這一類的合成軟體不提供除法(`/`)及取餘數(`%`)的運算。

```

wire [7:0] a, b, c, d;
assign a = c / d;
assign b = c % d;
  
```

4.1.3 不能用於電路合成的「邏輯閘型態」

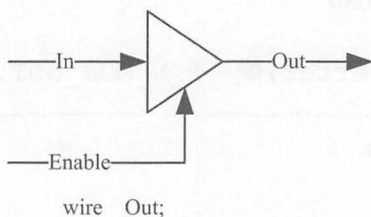
■ `tranif1`、`tranif0`、`rtran`、`rtranif1`、`rtranif0` 等邏輯閘型態。

■ `triand`、`trior`、`tri1`、`tri0`、`triereg` 等接線型態。

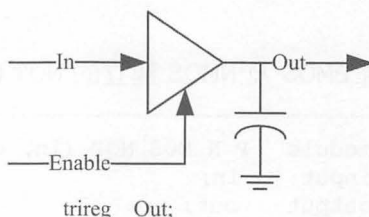
■ `nmos`、`pmos`、`cmos`、`rnmos`、`rpmos`、`rcmos` 等元件。

■ `supply1` (Power net)、`supply0` (Ground net) 等訊號改變元件。

例 1 wire 與 `triereg` (net with capacitive storage) 的差異

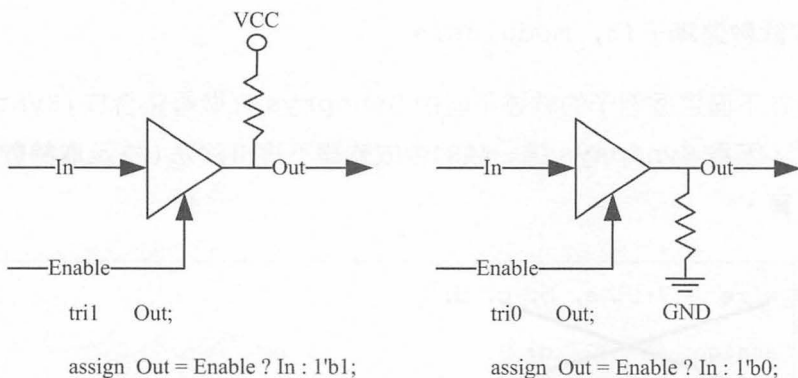


`assign Out = Enable ? In : 1'bz;`

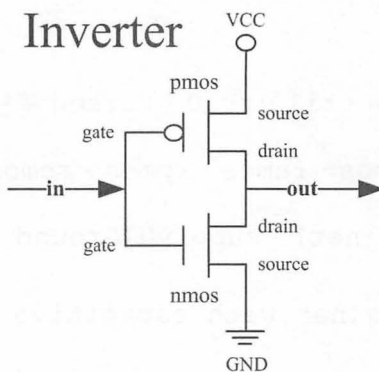


`assign Out = Enable ? In : Out;`

例 2 tri1(net with weak pull up device) 與 tri0(net with weak pull down device) 的差異



例 3 用 PMOS (P-channel Metal Oxide Semiconductor, P 通道金屬氧化物半導體) 及 NMOS (N-channel Metal Oxide Semiconductor, N 通道金屬氧化物半導體) 來建立反向器 NOT (Inverter) 閘



■ 用 PMOS 及 NMOS 建立的 NOT (Inverter) 閘, P_N_MOS_NOT.V

```

module P_N_MOS_NOT (in, out);
input  in;
output out;
wire  out;
  
```

```

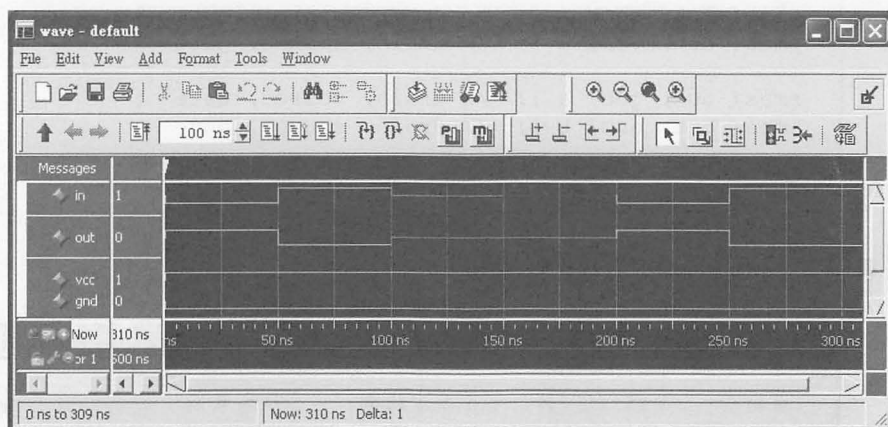
supply1  vcc;
supply0  gnd;

    pmos( out, vcc, in ); // drain, source, gate
    nmos( out, gnd, in ); // drain, source, gate

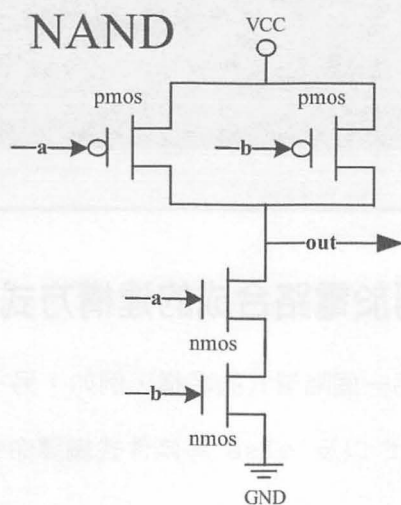
endmodule

```

■ P_N_MOS_NOT 的模擬結果



例 4 用 PMOS 及 NMOS 來建立 NAND 閘



- 用 PMOS 及 NMOS 來建立 NAND 閘，P_N_MOS_NAND.V

```
module P_N_MOS_NAND (a, b, out);
input a, b;
output out;
wor out; // 必須宣告成 wor 型態

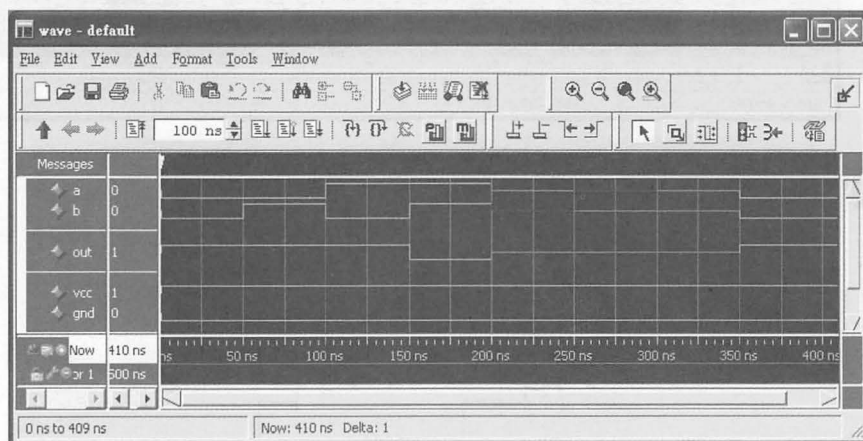
supply1 vcc;
supply0 gnd;

pmos( out, vcc, a ); // drain, source, gate
pmos( out, vcc, b ); // drain, source, gate

nmos( out, gnd, a ); // drain, source, gate
nmos( out, gnd, b ); // drain, source, gate

endmodule
```

- P_N_MOS_NAND 的模擬結果



4.1.4 其他不能用於電路合成的建構方式

- 不能在模組內有另一個階層式的名稱，例如：另一個模組 (module)。
- `ifdef、`endif 以及 `else 等條件式編譯命令 (Compiler Directives)。



4.2 能用於電路合成的 Verilog 語法

以下這些 Verilog 的「敘述」、「運算子」以及「邏輯閘」不僅能用於電路的模擬 (Simulation) 也能用於電路合成 (Synthesis) 的描述電路中。

4.2.1 能用於電路合成的「敘述」

除了上一節列出的不能用於合成的「敘述」，其餘的「敘述」都可以用於電路的合成。如下：

- ``include` 敘述。
- ``define` 敘述。
- `module`、`endmodule` 敘述。
- `parameter` 敘述。
- `input`、`output`、`inout` 敘述。
- `wire`、`wand`、`wor` 敘述。
- `reg` 敘述。
- `begin`、`end` 敘述。
- `always` 敘述。
- `assign` 敘述。
- `@`、`posedge`、`negedge` 敘述。
- `for` 敘述。
- `if`、`else` 敘述。
- `case`、`casex`、`casez` 敘述。

- function 敘述。
- task 敘述。
- defparam、#敘述。

4.2.2 能用於電路合成的「運算子」

除了上一節列出的不能用於合成的「運算子」，其餘的「運算子」都可以用於電路的合成。

(一) 二元逐位元運算子 (Binary Bit-wise operators)

二元逐位元運算子，包括：(1) $\sim$ ，Bit-wise NOT、(2) $\&$ ，Bit-wise AND、(3) $|$ ，Bit-wise OR、(4) $\wedge$ ，Bit-wise XOR，以及 (5) $\sim\wedge$ 或 $\wedge\sim$ ，Bit-wise XNOR 等 5 種。

- $\sim$ ，Bit-wise NOT

真值表

b	0	1
$\sim b$	1	0

對於特定一筆輸入資料的任何一個位元：若第 n 個位置為 0，則運算結果的第 n 個位置為 1；若第 n 個位置為 1，則運算結果的第 n 個位置為 0。

例：assign a = $\sim b$;

若 b = 4'b0010

則 a = 4'b1101

■ &, Bit-wise AND

真值表

b	0	0	1	1
c	0	1	0	1
b & c	0	0	0	1

對於特定二筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 0，則運算結果的第 n 個位置為 0；若第 n 個位置皆為 1，則運算結果的第 n 個位置為 1。

例：assign a = b & c;

若 b = 4'b0011

& c = 4'b1010

則 a = 4'b0010

■ |, Bit-wise OR

真值表

b	0	0	1	1
c	0	1	0	1
b c	0	1	1	1

對於特定二筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 1，則運算結果的第 n 個位置為 1；若第 n 個位置皆為 0，則運算結果的第 n 個位置為 0。

例：assign a = b | c;

若 b = 4'b0011

| c = 4'b1010

則 a = 4'b1011

■ ^, Bit-wise XOR

真值表

b	0	0	1	1
c	0	1	0	1
b ^ c	0	1	1	0

對於特定二筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 1 另一個為 0，則運算結果的第 n 個位置為 1；若第 n 個位置皆為 1 或 0，則運算結果的第 n 個位置為 0。

Bit-wise XOR 和 Bit-wise XNOR 有互補的關係。

例：assign a = b ^ c;

若 b = 4'b0011

^ c = 4'b1010

則 a = 4'b1001

■ ~^ 或 ^~, Bit-wise XNOR

真值表

b	0	0	1	1
c	0	1	0	1
b ~^ c	1	0	0	1

對於特定二筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 1 另一個為 0，則運算結果的第 n 個位置為 0；若第 n 個位置皆為 1 或 0，則運算結果的第 n 個位置為 1。

Bit-wise XNOR 和 Bit-wise XOR 有互補的關係。

例：assign a = b ~^ c; 或 assign a = b ^~ c;

若 b = 4'b0011

~^c = 4'b1010

則 a = 4'b0110

(二) 單元運算子 (Unary reduction operators)

單元運算子，包括：(1) &, reduction AND、(2) ~&, reduction NAND、(3) |, reduction OR、(4) ~|, reduction NOR、(5) ^, reduction XOR，以及 (6) ~^ 或 ^~, reduction XNOR 等 6 種。

■ &, reduction AND

真值表

b	0	1	00	01	10	11
&b	0	1	0	0	0	1

對於特定一筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 0，則運算結果為 0；若第 n 個位置皆為 1，則運算結果為 1。

reduction AND 和 reduction NAND 有互補的關係。

例：assign a = & b;

若 b = 4'b0000，則 a = 1'b0

若 b = 4'b0111，則 a = 1'b0

若 b = 4'b1111，則 a = 1'b1

■ ~&, reduction NAND

真值表

b	0	1	00	01	10	11
~&b	1	0	1	1	1	0

對於特定一筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 0，則運算結果為 1；若第 n 個位置皆為 1，則運算結果為 0。

reduction NAND 和 reduction AND 有互補的關係。

例：assign a = ~& b;

若 $b = 4'b0000$ ，則 $a = 1'b1$

若 $b = 4'b0111$ ，則 $a = 1'b1$

若 $b = 4'b1111$ ，則 $a = 1'b0$

■ |, reduction OR

真值表

b	0	1	00	01	10	11
b	0	1	0	1	1	1

對於特定一筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 1，則運算結果為 1；若第 n 個位置皆為 0，則運算結果為 0。

reduction OR 和 reduction NOR 有互補的關係。

例：assign a = | b;

若 $b = 4'b0000$ ，則 $a = 1'b0$

若 $b = 4'b0111$ ，則 $a = 1'b1$

若 $b = 4'b1111$ ，則 $a = 1'b1$

■ ~|, reduction NOR

真值表

b	0	1	00	01	10	11
~ b	1	0	1	0	0	0

對於特定一筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 1，則運算結果為 0；若第 n 個位置皆為 0，則運算結果為 1。

reduction NOR 和 reduction OR 有互補的關係。

例：assign a = ~| b;

若 b = 4'b0000，則 a = 1'b1

若 b = 4'b0111，則 a = 1'b0

若 b = 4'b1111，則 a = 1'b0

■ ^, reduction XOR

真值表

b	0	1	00	01	10	11
^b	0	0	0	1	1	0

若輸入資料的位元數為 1 個，則運算的結果為 0，相當於是清除的動作。

若輸入資料的位元數為 2 個(含)以上，且輸入資料值含有奇數個 1，則運算的結果為數 1，否則必為 0。

通常用於算術邏輯運算單元(ALU)中的 Odd(奇同位元)判斷旗標(flags)訊號。

reduction XOR 和 reduction XNOR 有互補的關係。

例：assign a = ^ b;

若 b = 4'b0000，則 a = 1'b0

若 b = 4'b0111，則 a = 1'b1

若 b = 4'b1111，則 a = 1'b0

■ ~^ 或 ^~, reduction XNOR

真值表

b	0	1	00	01	10	11
~^b	1	1	1	0	0	1

若輸入資料的位元數為 1 個，則運算的結果為 1，相當於是設定的動作。

若輸入資料的位元數為 2 個(含)以上、且輸入資料值含有偶數個 1，則運算的結果為數 1，否則必為 0。

通常用於算術邏輯運算單元(ALU)中的 Even(偶同位元)判斷旗標(flags)訊號。

reduction XNOR 和 reductionXOR 有互補的關係。

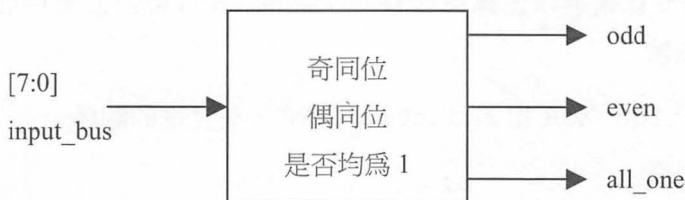
例：assign a = ~^ b; 或 assign a = ^~ b;

若 b = 4'b0000，則 a = 1'b1

若 b = 4'b0111，則 a = 1'b0

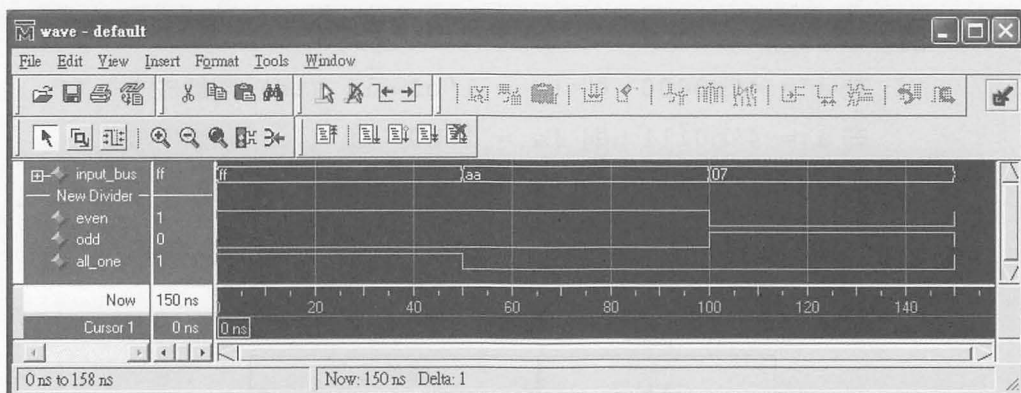
若 b = 4'b1111，則 a = 1'b1

BitWise.V：單元運算子應用例子，判別輸入的資料是否為奇同位、偶同位及是否均為 1。



```
// BitWise.V: 判別輸入的資料是否為奇同位、偶同位及是否均為 1
module BitWise(input_bus , even, odd, all_one);
input    [7:0] input_bus;
output   even, odd, all_one;
wire     even, odd, all_one;

    assign odd = ^ input_bus; // 奇同位的判別
    assign even = ~^ input_bus; // 偶同位的判別
    assign all_one = & input_bus; // 均為 1 的判別
endmodule
```



▲ BitWise.V 的模擬結果

(三) 邏輯運算子 (Logical operators)

邏輯運算子，包括：(1)!, Logical NOT、(2)&&, Logical AND，以及(3)||, Logical OR等3種。

對於任何一個邏輯判斷而言：1 為判斷條件成立(真)、0 為判斷條件不成立(偽)。

■ !, Logical NOT

真值表

條件	經過 ! 後
不成立	成立
成立	不成立

a	0	1	00	01	10	11
!a	1	0	1	0	0	0

對於特定一筆輸入資料的任何一個位元：若第 n 個位置只要有一個為 1，則運算結果為 0；若第 n 個位置皆為 0，則運算結果為 1。

例：if (!a)

若 $a = 4'b0000$ ，則 $!a = 1'b1$ 、if (!a) 判斷成立。

若 $a = 4'b0111$ ，則 $!a = 1'b0$ 、if (!a) 判斷不成立。

若 $a = 4'b1111$ ，則 $!a = 1'b0$ 、if (!a) 判斷不成立。

■ &&, Logical AND

真值表

&&		條件 1	
		不成立	成立
條件 2	不成立	不成立	不成立
	成立	不成立	成立

對於二個判斷條件(條件 1 及條件 2)：若這二個判斷條件皆成立，則結合判斷的結果為成立；若這二個判斷條件只要有一個是不成立，則結合判斷的結果為不成立。

例：if ((a>5) && (b<=2)) //如果 a>5 且 b<=2

若 $a=3'b000$ 且 $b=2'b11$ ，則 if ((a>5) && (b<=2)) 判斷不成立

若 $a=3'b000$ 且 $b=2'b01$ ，則 if ((a>5) && (b<=2)) 判斷不成立

若 $a=3'b110$ 且 $b=2'b11$ ，則 if ((a>5) && (b<=2)) 判斷不成立

若 $a=3'b110$ 且 $b=2'b01$ ，則 if ((a>5) && (b<=2)) 判斷成立

■ ||, Logical OR

		條件 1	
		不成立	成立
條件 2	不成立	不成立	成立
	成立	成立	成立

對於二個判斷條件(條件 1 及條件 2)：若這二個判斷條件皆不成立，則結合判斷的結果為不成立；若這二個判斷條件只要有一個是成立，則結

合判斷的結果為成立。

例：if ((a>5) || (b<=2)) //如果 a>5 或 b<=2

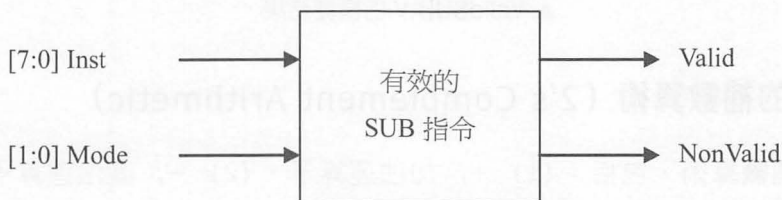
若 a=3'b000 且 b=2'b11，則 if ((a>5) && (b<=2)) 判斷不成立

若 a=3'b000 且 b=2'b01，則 if ((a>5) && (b<=2)) 判斷成立

若 a=3'b110 且 b=2'b11，則 if ((a>5) && (b<=2)) 判斷成立

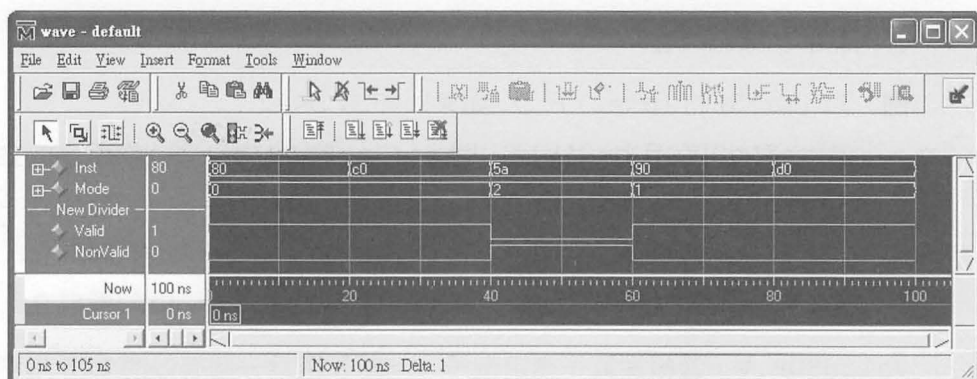
若 a=3'b110 且 b=2'b01，則 if ((a>5) && (b<=2)) 判斷成立

ValidSUB.V：邏輯運算子應用例子，判斷是否為一個有效的 SUB 指令



```

// ValidSUB.V：判斷是否為一個有效的 SUB 指令
// 其中，Valid 的定義為：
// 1. mode = IMMEDIATE 且 (inst = SUBA_imm 或 SUBB_imm)
// 2. mode = DIRECT 且 (inst == SUBA_dir 或 SUBB_dir)
// 其中，NonValid 的定義為：!Valid，即是對Valid取 Logical NOT
module Is_Valid_SUB_Inst(Inst, Mode, Valid, NonValid);
parameter IMMEDIATE = 2'b00, DIRECT = 2'b01;
parameter SUBA_imm = 8'h80, SUBA_dir = 8'h90,
          SUBB_imm = 8'hc0, SUBB_dir = 8'hd0;
input [7:0] Inst;
input [1:0] Mode;
output Valid, NonValid;
wire Valid, NonValid;
assign Valid = (
  ( (Mode == IMMEDIATE) &&
    ((Inst == SUBA_imm) || (Inst == SUBB_imm))) ||
  ( (Mode == DIRECT) &&
    ((Inst == SUBA_dir) || (Inst == SUBB_dir)) )
);
assign NonValid = !Valid;
endmodule
  
```



▲ ValidSUB.V 的模擬結果

(四) 2 的補數算術 (2's Complement Arithmetic)

2 的補數算術，包括：(1) +，加法運算子、(2) -，減法運算子，以及 (3) *，乘法運算子等 3 種。

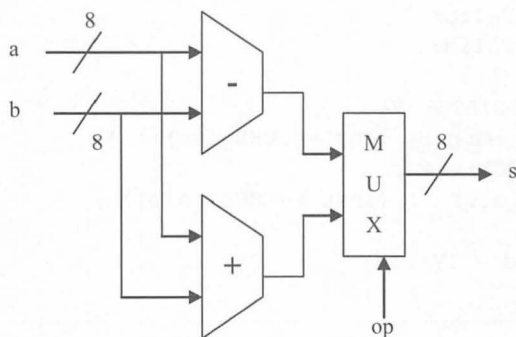
■ +，加法運算子

例：assign s = a + b;

■ -，減法運算子

例 1：assign s = a - b;

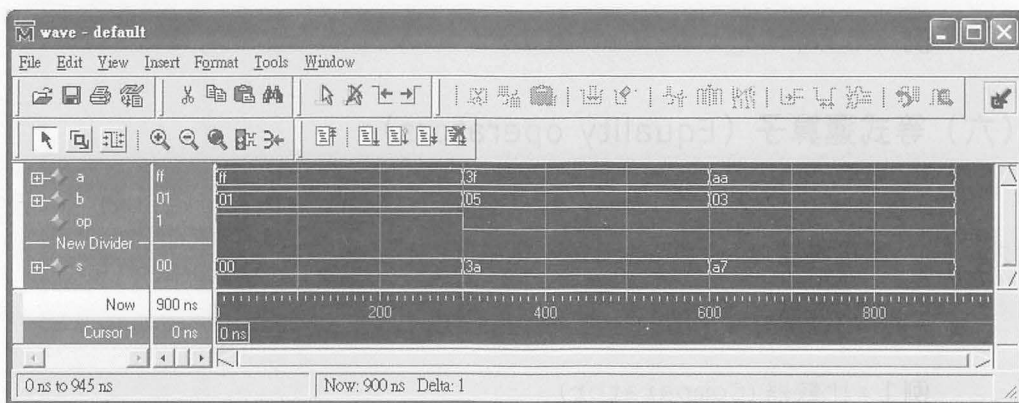
例 2：加減法 (Do Add or Subtract)



```
// AddOrSub.V:加減法 (Do Add or Subtract)
// 如果 op 等於 ADD, 則 s = a + b
// 如果 op 不等於 ADD, 則 s = a - b
module Add_or_Subtract(a, b, op, s);
parameter SIZE = 8;
parameter ADD = 1'b1;
input op;

input [SIZE-1:0] a, b;
output [SIZE-1:0] s;

assign s = (op == ADD) ? a+b : a-b;
endmodule
```



▲ AddOrSub.V 的模擬結果

■ *, 乘法運算子

例: assign s = a * b;

(五) 關係運算子 (Relational operators)

關係運算子，包括：(1) >, 大於運算子、(2) <, 小於運算子、(3) >=, 大於等於運算子，以及 (4) <=, 小於等於運算子等 4 種。

能用 >= 就不要用 >、電路合成後 >= 的邏輯電路比 > 簡單，能用 <= 就不要用 <、電路合成後 <= 的邏輯電路比 < 簡單。

■ >, 大於運算子

例: wire a_bigger = a > b;

■ <, 小於運算子

例: wire a_less = a < b;

■ >= , 大於等於運算子

例: wire a_gte = a >= b;

■ <=, 小於等於運算子

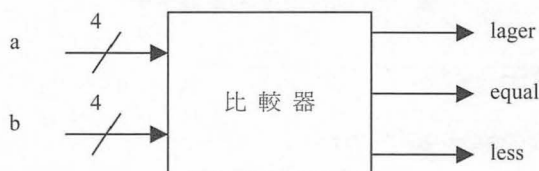
例: wire a_lte = a <= b;

(六) 等式運算子 (Equality operators)

等式運算子，包括：(1) ==, 相等運算子(Logical equality)，以及 (2) !=, 不相等運算子(Logical inequality)等 2 種。

■ ==, 相等運算子(Logical equality)

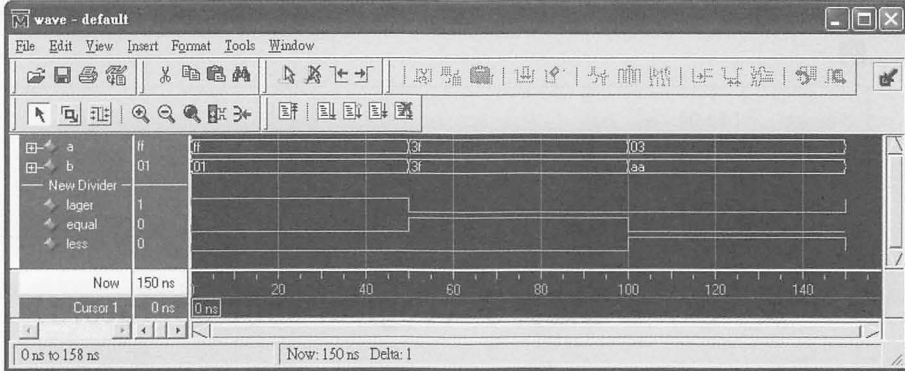
例 1: 比較器 (Comparator)



```
// Compare.V: 比較器
module Comparator (a, b, lager, equal, less);
parameter size = 8;
input [size-1:0] a, b;
output lager, equal, less;
wire lager, equal, less;

assign lager = (a > b);
assign equal = (a == b);
```

```
assign less = (a < b);
endmodule
```

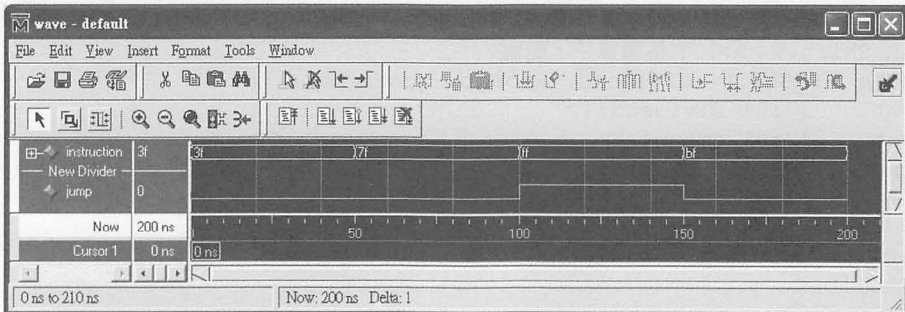


▲ Compare.V 的模擬結果

例 2：「位元截取」範例

```
// PartBit.V: 如果 instruction 的第 7 及第 6 位元均為 1, 也就是 2'b11
//            和 JMP = 2'h3 相等, 則令 jump 等於 1
//            否則令 jump 等於 0
module Is_Jump_Instruction(instruction, jump);
parameter JMP = 2'h3;
input      [7:0] instruction;
output     jump;
wire       jump;

assign     jump = (instruction[7:6] == JMP);
endmodule
```



▲ PartBit.V 的模擬結果

- !=, 不相等運算子 (Logical inequality)

以上的這些關係運算子都可用於向量式 (Vector) 的處理。

例：

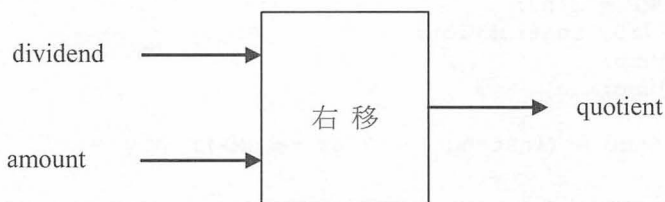
```
input    [7:0] a, b;
wire     a_bigger = a > b;
wire     eq = (a==b);
wire     a_less = a < b;
```

(七) >> 及 <<, 移位運算子 (Logical shift operators)

移位運算子，包括：(1) >>, 右移運算子 (Right Shift)，以及 (2) <<, 左移運算子 (Left Shift) 等 2 種。

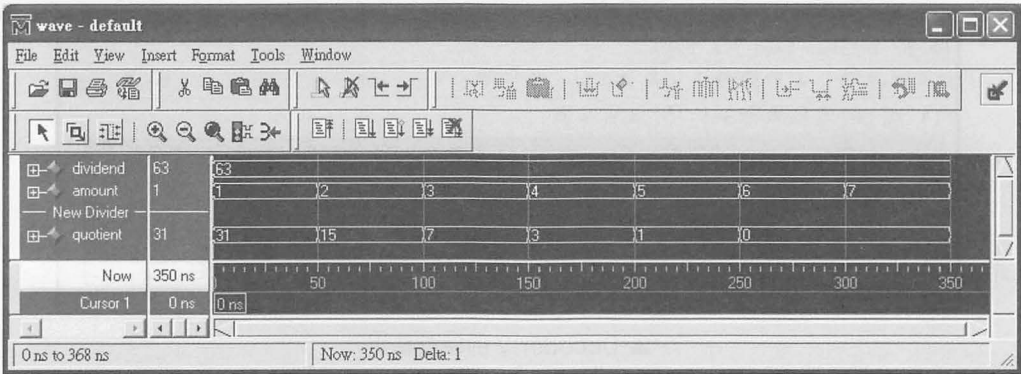
- >>, 右移運算子 (Right Shift)

例：將 dividend 右移 amount 次，每右移 1 次相當於是將 dividend 除以 2。



```
// R_Shift.V:>>, 右移運算子，範例
module R_Shift (dividend, amount, quotient);
input    [7:0] dividend;
input    [2:0] amount;
output   [7:0] quotient;
wire     [7:0] quotient;

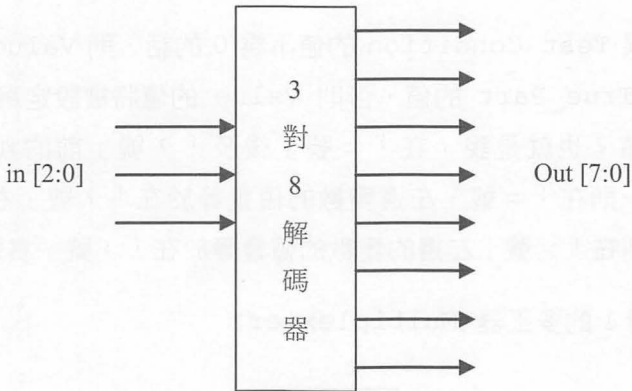
    assign quotient = dividend >> amount;
endmodule
```



▲ R_Shift.V 的模擬結果

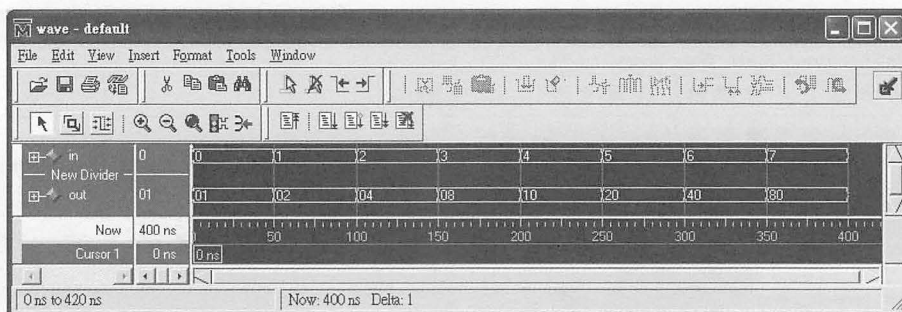
■ <<, 左移運算子 (Left Shift)

例：解碼器 (Decoder)



```
// Decoder.V: 3 對 8 的解碼器
module Decoder(in, Out);
input  [2:0] in;
output [7:0] Out;
wire    [7:0] Out;

    assign Out = 1'b1 << in;
endmodule
```

▲ Decoder.V 的模擬結果

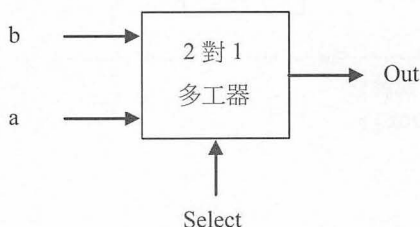
(八) ?:, 條件運算子 (Conditional operators)

Verilog 只有 ?: 這個條件運算子。

表示法: Value = Test _Condition ? True_Part : False_Part;

說明: 如果 Test Condition 的值不為 0 的話，則 Value 的值將被設定為 True_Part 的值，否則 Value 的值將被設定為 False_Part 的值。也就是說：在「= 號」後及「? 號」前的判斷如果成立的話，則在「= 號」左邊變數的值會等於在「? 號」右邊變數的值、否則在「= 號」左邊的變數的值會等於在「: 號」右邊變數的值。

例 1: 2 對 1 的多工器 (Multiplexier)



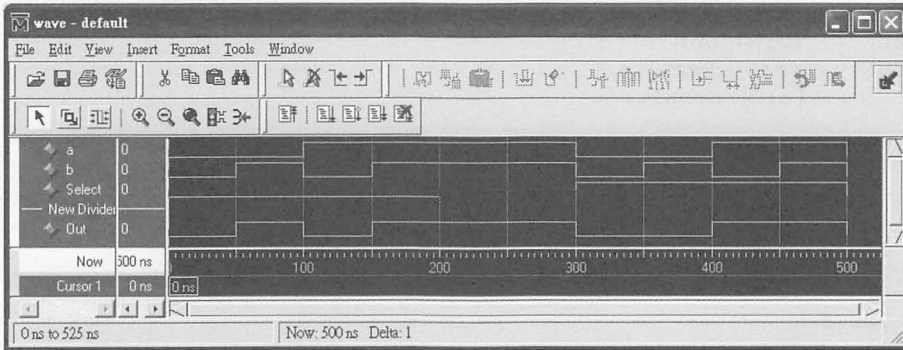
```
// Mux2to1.V: 2 對 1 的 Multiplexier
// 若 Select = 1, 則 Out = a, 否則 Out = b
module Mux2to1 (a, b, Select, Out);
```



```
input    a, b, Select;
output   Out;

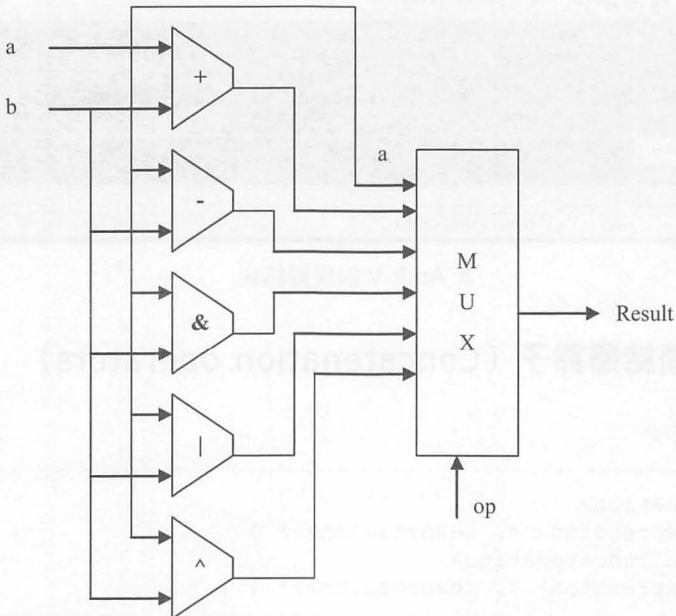
wire     Out;

    assign Out = Select ? a : b;
endmodule
```



▲ Mux2to1.V 的模擬結果

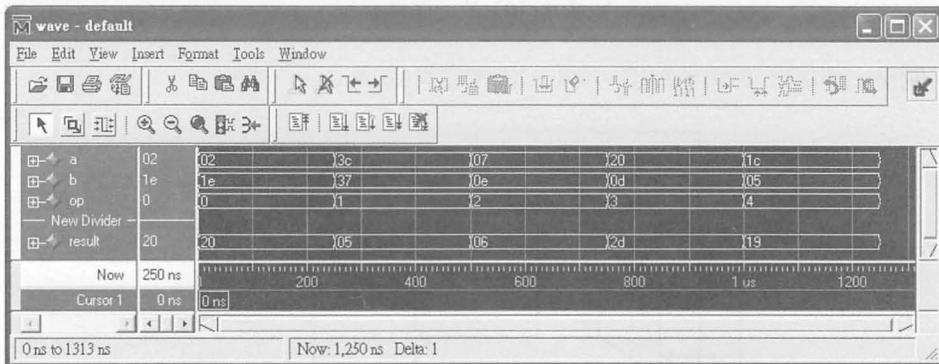
例 2：ADD、SUB、AND、OR、XOR 運算範例



```
// Arith.V: 算術運算
// 若 op = ADD, 則 Result = a + b; 若 op = SUB, 則 Result = a - b;
// 若 op = AND, 則 Result = a & b; 若 op = OR, 則 Result = a | b;
// 若 op = XOR, 則 Result = a ^ b;
// 若 op 不為上述 5 種之 1 的話, 則 Result = a;
module Arithmetic(a, b, op, Result);
parameter ADD = 3'h0, SUB = 3'h1, AND = 3'h2,
          OR = 3'h3, XOR = 3'h4;
input      [5:0] a, b;
input      [2:0] op;
output     [6:0] Result;
wire       [6:0] Result;

assign Result = ( (op == ADD) ? a + b : (
                    (op == SUB) ? a - b : (
                    (op == AND) ? a & b : (
                    (op == OR) ? a | b : (
                    (op == XOR) ? a ^ b : (a) ))))
);

endmodule
```



▲ Arith.V 的模擬結果

(九) {}, 連結運算子 (Concatenation operators)

BNF 表示法：

```
<concatenation>
::= { <expression> <, <expression>>* }
<multiple_concatenation>
::= { <expression> <, <expression>>* } }
```

說明：用以連結二個或以上的值到一個變數中，或將一個變數的值劃分到二個或以上的變數中。

例 1：

```

wire      [3:0] temp;
wire      [3:0] nibble1, nibble2;
wire      [7:0] Result1, Result2;

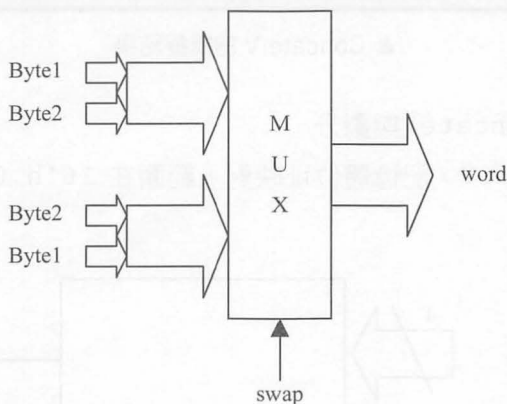
// 「連結」
assign    Result1 = {2'b1x, 6'hf}; // Result1 = 8'b1x001111;

assign    {nibble1, nibble2} = Result1; // 「劃分」

assign    temp = 4'b1010;
// 「連結」
assign    Result2 = {temp, temp}; // Result2 = 8'b10101010;

```

例 2：連結 (Concatenate)

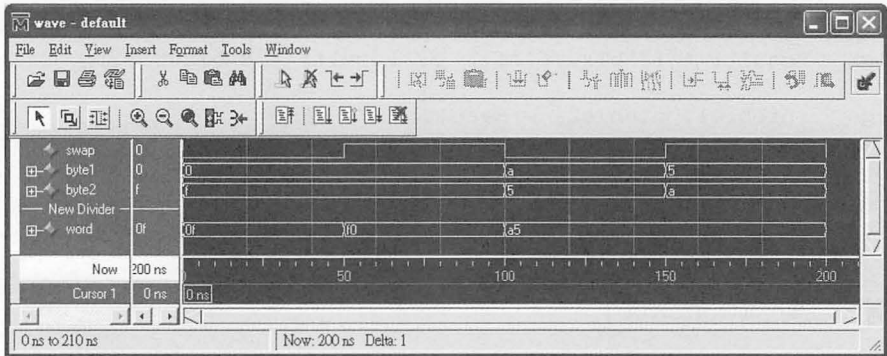


```

// Concat.V
// 如果 swap = 1, 則 word = byte2 連結 byte1
// 否則 word = byte1 連結 byte2
module    Concat (swap, byte1, byte2, word);
input     swap;
input     [3:0] byte1;
input     [3:0] byte2;
output    [7:0] word;
reg       [7:0] word;

```

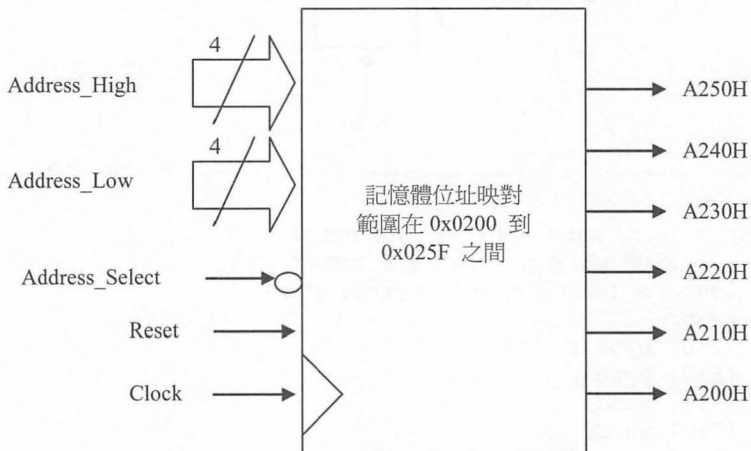
```
always @(swap or byte1 or byte2)
begin
    if(swap)
        word = {byte2, byte1}; // word = byte2 << 4 + byte1;
    else
        word = {byte1, byte2}; // word = byte1 << 4 + byte2;
    end
Endmodule
```



▲ Concat.V 的模擬結果

例 3：連結 (Concat) 與劃分

Mem_Map.V：記憶體位址映對，範圍在 16'h 0200 到 16'h025F 之間。



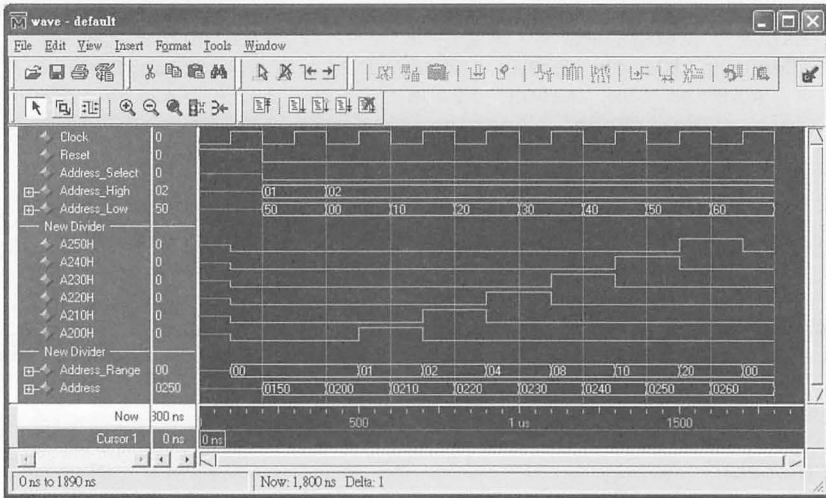
```

// Mem_Map.V: 記憶體位址映對, 範圍在 16'h 0200 到 16'h025F 之間
module Mem_Map (Clock, Reset, Address_Select, Address_High,
                Address_Low, A250H, A240H, A230H, A220H, A210H, A200H);
input          Clock, Reset, Address_Select;
input          [7:0] Address_High, Address_Low;
output         A250H, A240H, A230H, A220H, A210H, A200H;
reg            [5:0] Address_Range;
// 將 Address_High 及 Address_Low 「連結」 為一個 Address 匯流排
wire          [15:0] Address = {Address_High, Address_Low};

// 將 Address_Range 「劃分」 成獨立的輸出訊號
assign {A250H, A240H, A230H, A220H, A210H, A200H} = Address_Range;

always @(posedge Clock)
begin
    if (Reset)
    begin
        Address_Range = 0;
    end
    else
    begin
        if (Address_Select == 1'b0)
        begin // 預設為 0: Address 不在 16'h 0200 到 16'h025F 之間
            Address_Range = 0;
            if ((Address >= 16'h0200) && (Address < 16'h0210))
                Address_Range[0] = 1; // A200H = 1
            else
                if ((Address >= 16'h0210) && (Address < 16'h0220))
                    Address_Range[1] = 1; // A210H = 1
                else
                    if ((Address >= 16'h0220) && (Address < 16'h0230))
                        Address_Range[2] = 1; // A220H = 1
                    else
                        if ((Address >= 16'h0230) && (Address < 16'h0240))
                            Address_Range[3] = 1; // A230H = 1
                        else
                            if ((Address >= 16'h0240) && (Address < 16'h0250))
                                Address_Range[4] = 1; // A240H = 1
                            else
                                if ((Address >= 16'h0250) && (Address < 16'h0260))
                                    Address_Range[5] = 1; // A250H = 1
                                else
                                    Address_Range = 0;
                            end
                        end
                    end
                end
            end
        end
    end
end
endmodule

```



▲ Mem_Map.V 的模擬結果

(十) {{}}, 重覆運算子 (Duplication operators)

BNF 表示法：

```
{n{ <expression> <, <expression>}* }}
```

說明：用以重覆產生 n 個 <expression>。

例 1：

```
wire    [7:0] Result1, Result2, Result3;
wire    [1:0] temp;

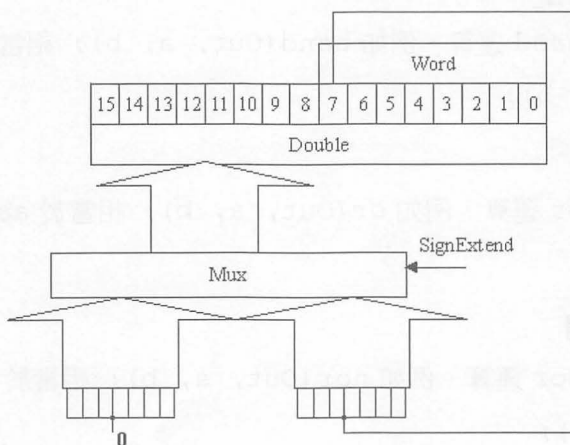
assign  Result1 = {2{4'b110x}}; // Result1 = 8'b110x110x;
assign  Result2 = {3{2'b1x}, 2'b00}; // Result2 = 8'b1x1x1x00;
assign  temp = 2'b01;
assign  Result3 = {2{2'b00}, 2'b11, temp}; // Result3 =
      8'b00001101;
```

例 2：符號位元展開 (Sign Extend) 的 Verilog 電路描述

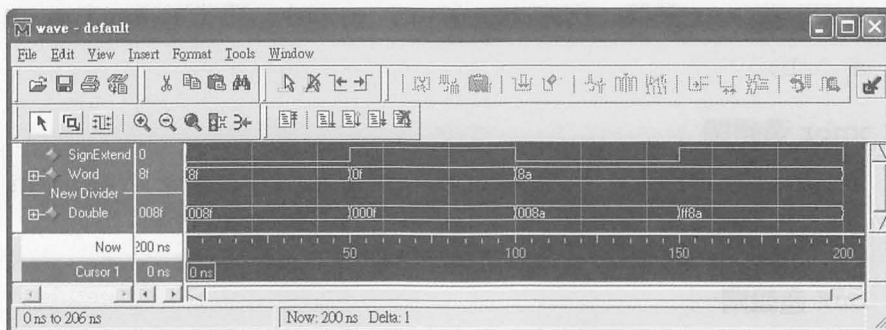
```
// Sign_Ext.V
// 如果 SignExtend = 1, 則 Double = 8 個 Word[7] + Word
```

```
//
// 否則 Double = 8 個 0 + Word
module Sign_Extend (SignExtend, Word, Double);
input SignExtend;
input [7:0] Word;
output [15:0] Double;
reg [15:0] Double;

always @(SignExtend or Word)
begin
    if (SignExtend)
        Double = {{8{Word[7]}}, Word}; // 重覆 8 個 Word[7] + Word
    else
        Double = {8'b0, Word}; // 重覆 8 個 0 + Word
    end
end
endmodule
```



▲ 符號位元展開 (Sign Extend) 的電路



▲ 符號位元展開 (Sign Extend) 的模擬結果

4.2.3 能用於電路合成的「邏輯閘」

能用於電路合成的「邏輯閘」，包括：and、nand、or、nor、xor、xnor、buf、bufif1、bufif0、not、notif1、notif0...等 Primitive Cells。

■ and 邏輯閘

a 與 b 做 and 運算，例如 `and(Out, a, b);`；相當於 `assign Out = a && b;`

■ nand 邏輯閘

a 與 b 做 nand 運算，例如 `nand(Out, a, b);`；相當於 `assign Out = ~(a && b);`

■ or 邏輯閘

a 與 b 做 or 運算，例如 `or(Out, a, b);`；相當於 `assign Out = a || b;`

■ nor 邏輯閘

a 與 b 做 nor 運算，例如 `nor(Out, a, b);`；相當於 `assign Out = ~(a || b);`

■ xor 邏輯閘

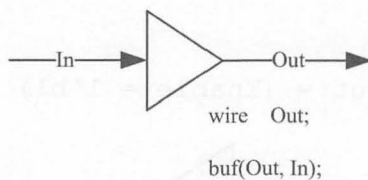
a 與 b 做 xor 運算，例如 `xor(Out, a, b);`；相當於 `assign Out = a ^ b;`

■ xnor 邏輯閘

a 與 b 做 xnor 運算，例如 `xnor(Out, a, b);`；相當於 `assign Out = a ~^ b;`

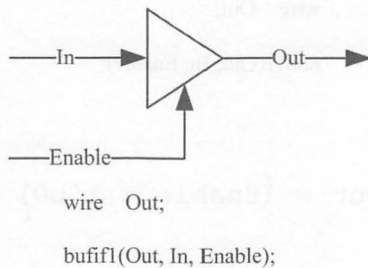
■ buf 邏輯閘

相當於 `assign Out = In;`



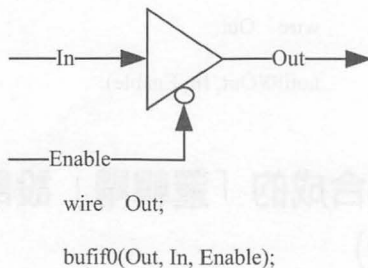
■ bufif1 邏輯閘

相當於 `assign Out = (Enable = 1'b1) ? In : 1'bz;`



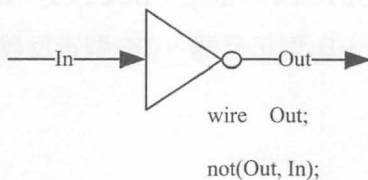
■ bufif0 邏輯閘

相當於 `assign Out = (Enable = 1'b0) ? In : 1'bz;`



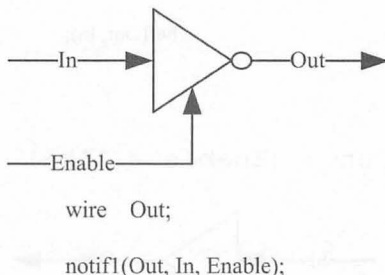
■ not 邏輯閘

相當於 `assign Out = ~In;`



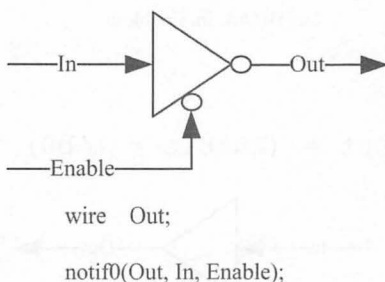
■ notif1 邏輯閘

相當於 `assign Out = (Enable = 1'b1) ? ~In : 1'bz;`



■ notif0 邏輯閘

相當於 `assign Out = (Enable = 1'b0) ? ~In : 1'bz;`

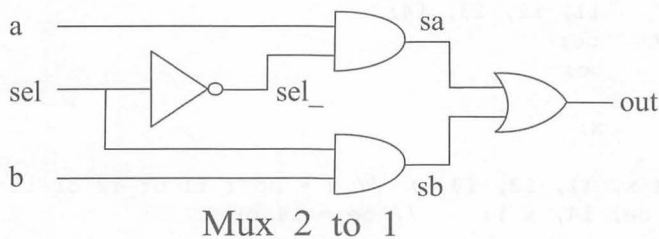


4.2.4 用可以電路合成的「邏輯閘」設計出電路圖 (Schematic)

還沒有 Verilog 這一類的硬體描述語言之前，數位邏輯電路中的電路圖就是所謂的 Schematic。Schematic 通常是由 and、nand、or、nor、xor、xnor、buf、bufif1、bufif0、not、notif1、notif0... 等 Primitive Cells 以及 Latch 鎖器、D 型正反器、JK 型正反器、T 型正反器... 等等畫出來的電路圖。

例 1 Mux_2_to_1 的電路圖 (Schematic)

當 $sel=0$ 時， $out=a$ ；當 $sel=1$ 時， $out=b$ 。



■ Mux_2_to_1 的 Verilog 程式碼

```

module  Mux_2_to_1 (a, b, sel, out);
input   a, b, sel;
output  out;
wire    out;

wire    sel_, sa, sb;

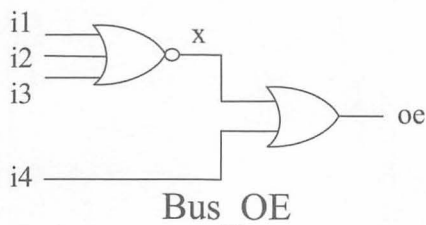
  not( sel_, sel );    // sel_ = not sel
  and( sa, a, sel_ );  // sa = a and sel_
  and( sb, b, sel_ );  // sb = b and sel_
  or( out, sa, sb );   // out = sa or sb

endmodule

```

例 2 Bus_OE 的電路圖 (Schematic)

當 $i1=0$ 、 $i2=0$ 且 $i3=0$ 時， $oe=1$ ；或是當 $i4=1$ 時， $oe=1$ ；其他情形時 $oe=0$ 。



■ Bus_OE 的 Verilog 程式碼

```
module Bus_OE (i1, i2, i3, i4, oe);  
input    i1, i2, i3, i4;  
output   oe;  
wire     oe;  
  
wire     x;  
  
    nor( x, i1, i2, i3 ); // x = not( i1 or i2 or i3 )  
    or( oe, i4, x );      // oe = i4 or x  
  
endmodule
```

本章練習**Question 1 :**

爲什麼需要有用不能用於電路合成的 Verilog 語法，例如：不能用於電路合成的「敘述」、「運算子」以及「邏輯閘型態」？

Question 2 :

爲什麼需要有用 `ifdef、`endif 以及 `else 等條件式編譯命令 (Compiler Directives) ？

Question 3 :

`if (a > 1)` 和 `if (a >= 2)` 這二個關係運算的結果是等效的？

`if (a > 1)` 和 `if (a >= 2)` 這二個關係運算，那一個電路合成後的面積會比較小？

Question 4 :

`if (a < 2)` 和 `if (a <= 1)` 這二個關係運算的結果是等效的？

`if (a < 2)` 和 `if (a <= 1)` 這二個關係運算，那一個電路合成後的面積會比較小？

PS Google 可以幫您找到解答。

